ML Network Intrusion Detector

Classifying network anomalies with scikit-learn on the NSL-KDD dataset

Cybersecurity Internship Project Report

Author: [Your Name] **Organisation:** [Internship Organisation] **Date:** November 2026

Abstract

This project builds a machine-learning network intrusion detection system (NIDS) that labels network connections as normal or malicious and identifies the attack family (DoS, Probe, R2L, U2R). Four scikit-learn models are compared on the public NSL-KDD benchmark: Logistic Regression, Random Forest, Gradient Boosting and an unsupervised Isolation Forest. Models trained on KDDTrain+ are evaluated on the official KDDTest+ set, which contains attack types unseen during training. Gradient Boosting performs best (F1 = 0.793, false-positive rate 2.8%, ROC-AUC 0.961). A central finding is that random-split cross-validation reports 99.9% F1 while the held-out test set yields about 75-79%, showing how strongly in-distribution validation overstates real-world IDS performance. The report also covers threshold analysis under a false-alarm budget, per-attack error analysis, explainability with permutation importance and SHAP, and a Streamlit demo application.

1. Introduction

Signature-based intrusion detection cannot recognise novel attacks, and security operations centres (SOCs) are overwhelmed by false alarms. Machine learning can learn the statistical footprint of malicious traffic and generalise to variants that signatures miss. The objectives of this project are:

- Build a reproducible, leakage-free ML pipeline for network intrusion detection.
- Compare supervised and unsupervised detectors using metrics that matter operationally, especially the false-positive rate.
- Measure generalisation to unseen attack types rather than relying on in-distribution accuracy.
- Explain model decisions and analyse which attacks are missed and why.
- Deliver a usable demo that scores uploaded connection records.

2. Dataset

NSL-KDD (Tavallaee et al., 2009) is a cleaned version of KDD Cup 99 with redundant records removed. Each record describes one network connection using 41 features: basic (duration, protocol, bytes), content (failed logins, root shell), time-window traffic statistics (e.g. count, serror_rate) and host-based statistics (dst_host_*). Attacks are grouped into four families: **DoS** (denial of service, e.g. neptune, smurf), **Probe** (scanning, e.g. nmap, portsweep), **R2L** (remote-to-local, e.g. guess_passwd) and **U2R** (user-to-root privilege escalation, e.g. buffer_overflow).

Split	Records	Purpose
KDDTrain+	125,973	Training and cross-validation
KDDTest+	22,544	Official held-out test; 17 attack types not seen in training (~16.6% of records)

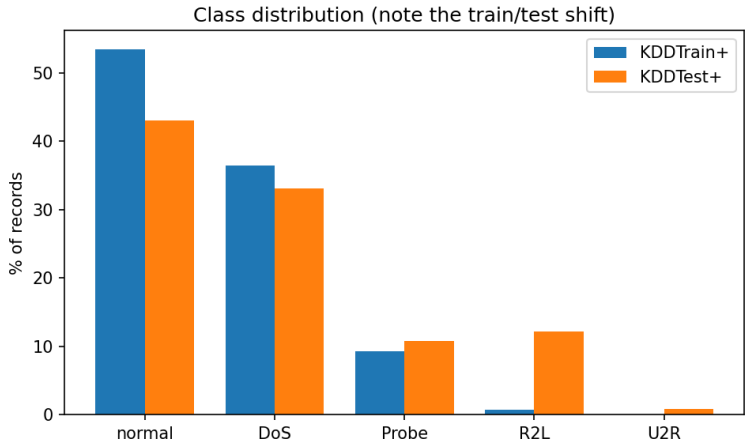


Figure 1. Class distribution shifts between training and test sets (R2L is far more common in the test set).

3. Methodology

3.1 Pre-processing

- **Cleaning:** duplicate removal, replacing infinite values with NaN, and dropping the constant column *num_outbound_cmds*.
- **Encoding and scaling:** median imputation and standardisation for numeric features; one-hot encoding for protocol_type, service and flag, with *handle_unknown=ignore* because the test set contains services absent from training.
- **No data leakage:** every transformation is fitted inside a scikit-learn *Pipeline* using training data only.
- **Class imbalance:** *class_weight="balanced"* for all supervised models.

3.2 Models

Model	Type	Notes
Logistic Regression	Supervised	Linear baseline
Random Forest	Supervised	200 trees
Gradient Boosting	Supervised	HistGradientBoostingClassifier
Isolation Forest	Unsupervised	Trained on normal traffic only; threshold at the 95th percentile of normal scores

3.3 Evaluation protocol

Models are trained on KDDTrain+ and tested on KDDTest+. Reported metrics are precision, recall (detection rate), F1, **false-positive rate (FPR)**, ROC-AUC and PR-AUC. FPR is emphasised because false alarms drive analyst fatigue in a SOC. A 5-fold stratified cross-validation on the training data is also reported to expose the gap between in-distribution and out-of-distribution performance.

4. Results

4.1 Binary detection (normal vs attack)

Model	Precision	Recall	F1	FPR	ROC-AUC	PR-AUC
Logistic Regression	0.917	0.625	0.744	7.5%	0.791	0.866
Random Forest	0.968	0.609	0.747	2.7%	0.961	0.964
Gradient Boosting	**0.969**	**0.671**	**0.793**	**2.8%**	**0.961**	**0.967**
Isolation Forest (unsup.)	0.955	0.655	0.777	4.1%	0.941	0.953

Table 1. Test-set results at the default 0.5 decision threshold (Isolation Forest uses its calibrated threshold).

Gradient Boosting achieves the best F1 (0.793). Random Forest and Gradient Boosting both keep the false-positive rate below 3%, versus 7.5% for the Logistic Regression baseline. The unsupervised Isolation Forest, which never saw a labelled attack, reaches F1 = 0.777, which is notable for its ability to flag unknown threats.

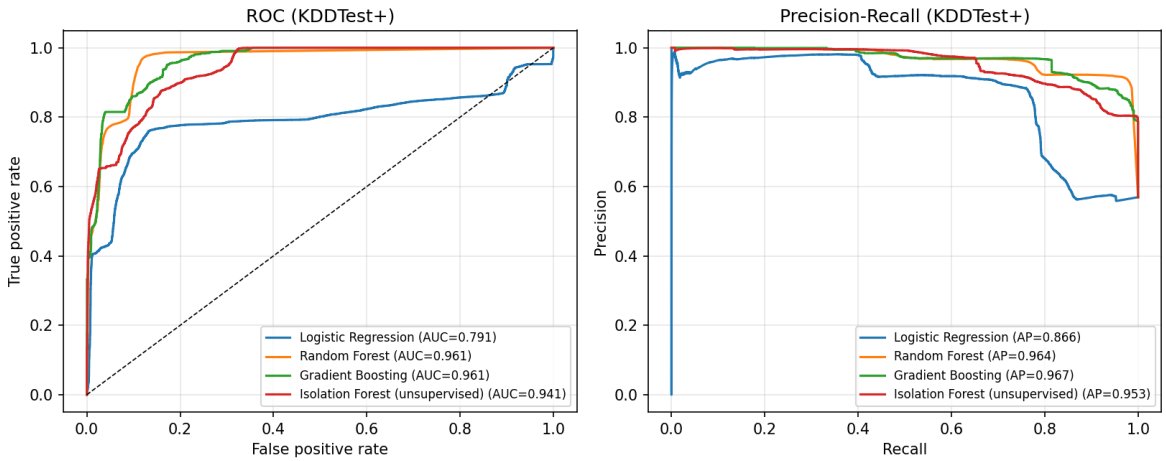


Figure 2. ROC and Precision-Recall curves on KDDTest+.

4.2 The generalisation gap

Five-fold cross-validation on the training data gives an F1 of **0.9988 +/- 0.0002** for Random Forest, whereas the same model scores **0.747** on KDDTest+. The ~25-point drop is caused by distribution shift: the test set contains new attack types and a different class mix. Random-split validation therefore gives a dangerously optimistic estimate of real-world detection performance.

4.3 Operating points: choosing a threshold by false-alarm budget

ROC-AUC is 0.96 but recall at the default 0.5 threshold is only 0.61-0.67. This shows the models rank traffic well, and the default cut-off is simply too conservative for shifted data. A SOC would set the threshold from an acceptable false-alarm budget:

Model	recall @ 1% FPR	recall @ 2% FPR	recall @ 5% FPR	recall @ 10% FPR
Random Forest	45.7%	51.3%	77.2%	89.0%
Gradient Boosting	47.5%	50.8%	81.5%	86.0%

Table 2. Detection rate (recall) at fixed false-positive rates on KDDTest+.

Allowing a 5% false-positive rate raises detection to 77-82%, and 10% FPR reaches 86-89%.

4.4 Attack-family classification

Class	Precision	Recall	F1	Support
normal	0.63	0.97	0.77	9,711
DoS	0.96	0.76	0.85	7,458
Probe	0.85	0.59	0.70	2,421
R2L	0.88	0.01	0.01	2,754
U2R	0.33	0.01	0.01	200

Table 3. Random Forest multiclass results on KDDTest+ (overall accuracy 0.74).

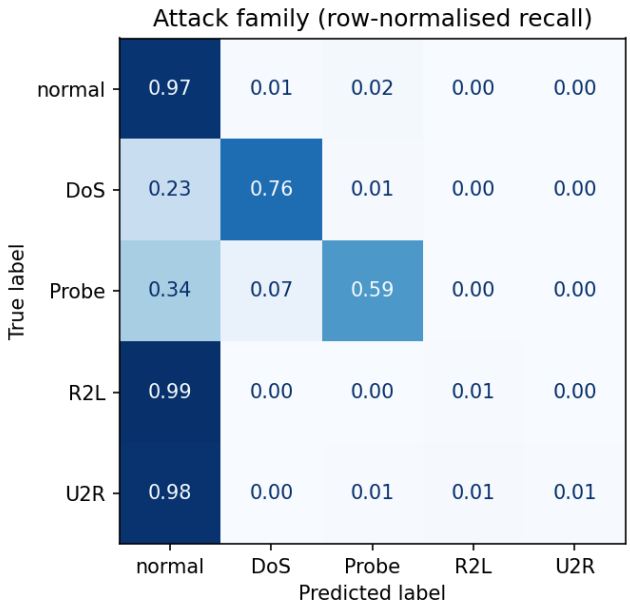


Figure 3. Row-normalised confusion matrix. R2L and U2R are almost entirely labelled as normal.

DoS and Probe attacks are detected well because they leave strong flow-level signatures (many connections, SYN errors, scanning patterns). R2L and U2R attacks are nearly invisible (recall 0.01) because they resemble legitimate sessions at the flow-statistics level and are rare in training data.

5. Error analysis

Per-attack detection rates for the Random Forest (attacks with at least 20 test records) show which threats slip through:

Attack	Test records	Detected
guess_passwd	1,231	0.0%
mailbomb	293	0.0%
snmpguess	331	0.0%
snmpgetattack	178	0.0%
processtable	685	1.8%
httptunnel	133	3.8%
buffer_overflow	20	10.0%
warezmaster	944	15.1%
apache2	737	18.0%
mscan	996	35.4%

Attack	Test records	Detected
pod	41	95.1%
back	359	97.8%
saint	319	98.7%
neptune	4,657	100.0%
portsweep	157	100.0%
ipsweep	141	100.0%
smurf	665	100.0%
satan	735	100.0%
nmap	73	100.0%

Table 4. Detection rate per attack type (Random Forest).

Volumetric and scanning attacks (neptune, smurf, ipsweep, portsweep, satan, nmap) are detected at or near 100%. Credential-guessing and application-layer attacks (*guess_passwd*, *snmpguess*, *snmpgetattack*, *mailbomb*, *processtable*, *apache2*) are missed almost completely. Several of these are absent from the training set (mailbomb, snmpguess, snmpgetattack, processtable, apache2, mscan), and the others generate traffic that looks statistically normal. Flow-level features are therefore insufficient for them; payload or host-level data would be needed.

6. Explainability

Permutation importance (drop in F1 when a feature is shuffled) and SHAP values were computed to verify that the model relies on security-meaningful signals. The most influential features were: *src_bytes*, *flag*, *dst_host_rerror_rate*, *dst_bytes*, *dst_host_srv_diff_host_rate*, *rerror_rate*.

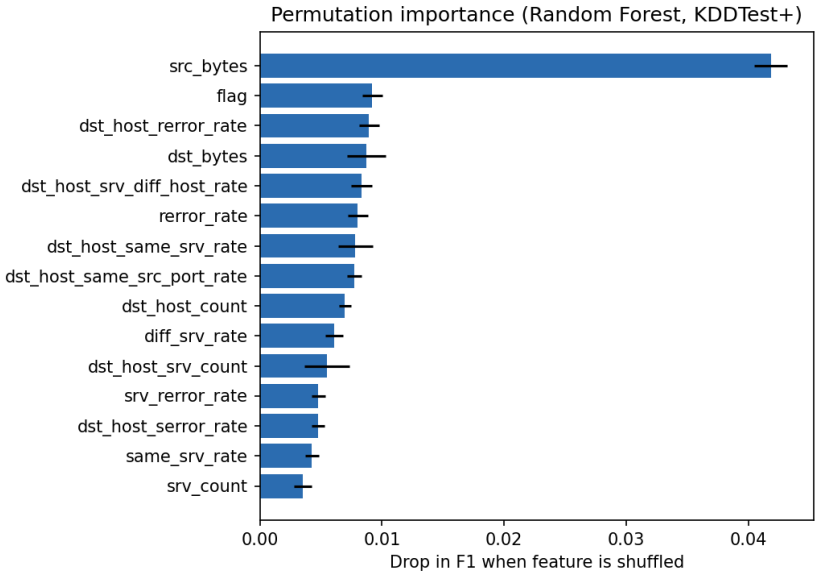


Figure 4. Permutation importance on KDDTest+ (original features).

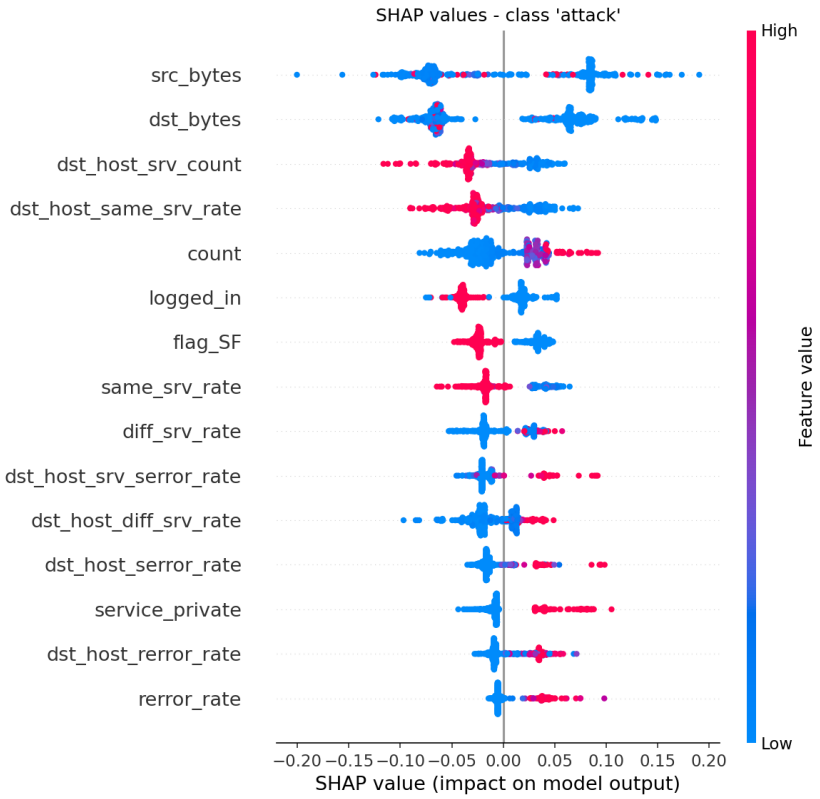


Figure 5. SHAP summary for the attack class. Each dot is a connection, colour shows feature value.

The SHAP plot is consistent with domain knowledge: a high connection *count* and high SYN-error rates (*dst_host_serror_rate*) push predictions toward 'attack' (flooding and scanning), while a successfully completed TCP session (*flag_SF*) and an authenticated session (*logged_in*) push toward 'normal'.

7. Demonstration application

A Streamlit web application loads the saved Random Forest models. Users upload a CSV of connection records (with or without a header), adjust the alert threshold with a slider, and see the number of alerts, alerts by predicted attack family, the most suspicious flows ranked by attack probability, and a download of all predictions. A command-line mode (*python ids_detector.py predict file.csv*) provides the same scoring in the single-file version.

8. Limitations

- NSL-KDD derives from 1999 DARPA data; modern traffic is largely encrypted and attack tooling has changed. Results show methodology rather than production readiness.
- R2L and U2R attacks are effectively undetected; flow statistics are not enough.
- The operating-point analysis uses the test ROC curve for illustration. A deployment must calibrate thresholds on validation data from the target network.
- Hyper-parameters were not extensively tuned, to avoid over-fitting to the test distribution.

9. Future work

- Repeat the study on CIC-IDS2017 and UNSW-NB15 and test cross-dataset transfer.
- Add cost-sensitive learning or oversampling (on training data only) for R2L/U2R.
- Capture live traffic with Scapy or Zeek and convert it to flow features for real-time detection.
- Map predicted categories to MITRE ATT&CK techniques and add concept-drift monitoring.

10. Conclusion

The project delivers a reproducible, leakage-free ML intrusion detector. Gradient Boosting gives the best detection (F1 0.793, FPR 2.8%), and an unsupervised Isolation Forest comes close without any attack labels. The most important lesson is methodological: random-split cross-validation reports 99.9% F1, but the realistic figure on unseen attacks is roughly 75-79%. Threshold selection by false-alarm budget, per-attack error analysis and explainability turn a raw accuracy number into an operationally meaningful assessment.

References

[1] M. Tavallaee, E. Bagheri, W. Lu, A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA), 2009.

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, 12, 2011.

[3] F. T. Liu, K. M. Ting, Z.-H. Zhou, "Isolation Forest," IEEE ICDM, 2008.

[4] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," NeurIPS, 2017.